

ElectronBot - In-Depth Project Analysis

Generated by multi-agent analysis (37 agents: 5 explorers per folder × 6 folders + 6 synthesizers + 1 global synthesis). Read-only — nothing was modified.

ElectronBot is the open-source desktop companion robot by **peng-zhihui** — a palm-sized figure, WALL-E/EVE style, with a **round 240×240 LCD face**, an embedded **USB camera**, and **6 degrees of freedom** (2 arms × roll+pitch, plus neck and waist).

The most important architectural idea: the robot itself is "dumb." All the intelligence — emoji/expression rendering, animation, motion planning — runs on a **host PC**. The STM32 firmware is just a thin bridge that pushes pixels to the LCD and relays angles to the servos. The face is video; the body motion is six floats; both travel over **a single USB cable**.

End-to-end data flow

1. **Unity Studio (PC, C#)** — authors/plays back 3D keyframes of pose+expression; computes 6 joint angles and chooses the face image/video per frame.
2. **UnityBridge.dll (C++)** — Unity P/Invokes 4 `Native_*` functions; pushes the image + 6 setpoints and returns measured joints + webcam frame (flipped).
3. **ElectronBotSDK-LowLevel.dll (C++)** — the single class (`ElectronLowLevel`) that owns the wire protocol. Each `Sync()` resizes the face to 240×240 RGB888 (172,800 bytes) and transmits it while reading telemetry.
4. **USBInterface.dll + libusb-win32 (BotDriver)** — raw USB bulk transport (NOT a virtual COM port).
5. **ElectronBot-fw on the STM32F405 (head board)** — receives the frame into a ping-pong buffer, DMAs it to the GC9A01 LCD via SPI1, and acts as I2C master of the 6 servos.
6. **ServoDrive-fw on the STM32F042 (one per joint)** — I2C slaves; each runs a 200 Hz position PID loop driving a brushed DC motor (H-bridge), reading a potentiometer via ADC for absolute angle feedback.

The **same 32-byte channel** that carries the 6 outgoing setpoints also returns the 6 measured angles every frame — face video and 6-DOF motion stay synchronized, and the PC can read back the robot when posed by hand ("physical-priority" mode).

1. Hardware — 5 Altium projects (2-layer, fab-ready)

Topology = **USB tree + a single shared I2C bus**.

Board	MCU	Role
BaseConnector	none	USB-C input (sink) on the rotating base. Connects to the body via an 8-way 0.5 mm flex FFC (the waist rotates).
SensorBoard	none	The torso's "electrical heart": HS8836A 4-port USB hub, PAJ7620U2 gesture sensor, MPU6050 IMU, 5 SH1.0-4P servo connectors. All on a shared I2C.
ElectronBot (head)	STM32F405RGT6 (M4F, 168 MHz, 1MB)	The brain. Round GC9A01 240×240 LCD via SPI, USB High-Speed via external PHY USB3300 (which is why it <i>has</i> to be an F4), CP2102 UART bridge, microSD slot. I2C master .
ServoDrive	STM32F042F6P6 (M0)	A board inside each modified RC servo + FM116B brushed DC H-bridge driver. I2C slave, reads pot via ADC, runs PID.
ServoDrive-DK	STM32F042	A debug twin electrically identical to ServoDrive, with pluggable headers for flashing/debugging on the bench.

⚠ **Do not confuse** ServoDrive (STM32F0 + brushed DC) with the head board (STM32F4). **There is no FOC/BLDC** anywhere. SensorBoard and BaseConnector have no firmware because they have no MCU.

2.Firmware — 3 STM32 projects (master/slave system)

ElectronBot-fw (head, STM32F405, M4F) — CubeMX/HAL-generated C in `Core/` + hand-written C++17 in `UserApp/ / Bsp/`. Boots HAL + FreeRTOS, but the real app is a **bare-metal super-loop** in `Main()` (FreeRTOS is just a launcher). Waits 2 s for the servos to boot (`HAL_Delay(2000)`), avoids locking up the I2C. Per frame: 4 iterations sending a 32-byte IN packet (with the 6 measured angles), busy-waits for the OUT image, DMAs an LCD stripe of $60 \times 240 \times 3 = 43,200$ bytes (RAMWR 0x2C on stripe 0, RAM-continue 0x3C afterwards), and after 4 stripes commands the 6 joints over I2C. DMA-SPI of one stripe overlaps the USB reception of the next via a **ping-pong buffer**. **I2C master** at the even addresses 2,4,6,8,10,12 (neck, L-roll, L-pitch, R-roll, R-pitch, waist).

ServoDrive-fw / ServoDrive-fw-II (joint, STM32F042, M0) — I2C slaves. A 200 Hz loop in the TIM14 ISR reads a **potentiometer** on ADC1_IN4/PA4 (NOT a magnetic encoder), maps calibrated counts ($250..3000 \rightarrow 0..180^\circ$), runs a position PID (the "DCE": $k_p \cdot \text{err} + k_i \cdot \int + k_v \cdot \int \text{vel} + k_d \cdot \Delta$, anti-windup), and drives the brushed DC motor as a sign-magnitude H-bridge on TIM3 (~48 kHz). I2C opcodes: 0x01 set angle, 0x02 velocity, 0x03 torque, 0x11/0x12 read angle/velocity, 0x21 set id, 0x22–0x25 PID gains, 0x26 torque limit, 0x27 home position, 0xff enable/disable; the response always echoes the current angle. Config persists in flash (emulated EEPROM).

The **two servo variants have byte-for-byte identical logic** (motor.cpp, DCE PID, I2C handler). They differ only in: (a) the peripheral layer — `ServoDrive-fw` = HAL; `ServoDrive-fw-II` = LL (smaller/faster, **changes I2C address live without a reboot**, adds an F030 target); (b) toolchain (CMake/CubelIDE vs PlatformIO).

3. Software — Layered host (PC) stack

- **USB driver (base):** `_Tools/BotDriver` installs **libusb-win32** → the robot enumerates as a **raw bulk device, not a COM port**. VID **0x1001** / PID **0x8023** (app); 0x0483/0x5740 (bootloader).
- **Transport:** `USBInterface.dll` (C-ABI over libusb0). Endpoints **EP1_OUT 0x01** (host→device), **EP1_IN 0x81** (device→host).
- **Core — ElectronBotSDK-LowLevel** : the `ElectronLowLevel` class that is the link. Protocol per `Sync()` :
- `SetImageSrc` → 240×240, BGRA→RGB = **172,800 bytes** into a ping-pong TX.
- `Sync()` sends the frame in **4 chunks paced by the MCU**. Per chunk: (1) **read 32 bytes from EP1_IN** (handshake + telemetry of the 6 angles), (2) write **84 × 512 B = 43,008 B** of image, (3) write a **224 B tail** (192 image + 32 extra-data). $4 \times (43,008 + 192) = 172,800$ = one frame.
- **32-byte extra block:** `byte[0]` = enable flag, `bytes[1..24]` = 6 little-endian floats (setpoints). The **short 224 B packet is the end-of-frame delimiter** that the firmware uses to flip the ping-pong.
- **Middle layer:**
- **Player** — plays images or MP4/MOV via OpenCV. ⚠ `SetPose / GetPose` are **empty stubs** (old LowLevel header without `SetJointAngles`) → **the Player only controls the screen, not the servos**.
- **UnityBridge** — `extern "C"` plugin with 4 entry points (`Native_OnInit/OnFixUpdate/OnKeyFrameChange/OnExit`); `OnFixUpdate` is non-blocking (detached thread).
- **Front-end 1 — Unity/ElectronBot-Studio** (Unity 2020.1.6f1, C#): 6-DOF on-screen rig via sliders; P/Invokes the 4 `Native_*` per `FixedUpdate` (~20 Hz). 3 modes: **0 = model-priority** (PC commands), **1 = disabled**, **2 = physical-priority** (reads the robot posed by hand). Keyframe authoring stores {6 angles, backlight, media}. `Unity/Release` ships Windows/Linux players v0.0.1.
- **Front-end 2 — _Tools/AHK-ExpansionPack** (AutoHotkey v1): a separate build of the SDK (OpenCV 4.5.5). Joint limits: head ±15, waist ±90, open arm ±30, raise arm ±180. 8 demos (WeChat control, face mirror, voice, etc.).
- **Calibration — _Tools/ServoToolkit** : Qt5 GUI via **libusb-1.0**. **Requires flashing its own firmware** (`ElectronBot-fw-servo-toolkit.hex`) onto the head, temporarily replacing the app; connect **one servo at a time** when addresses are not yet set.

⚠ **Version drift:** CMake points to OpenCV 3.4.8 (`opencv_world348d`, debug) but the shipped DLLs are 4.5.5; CMakeLists have hardcoded paths `D:\Libraries\Cpp\...`.

4. CAD-Model — Mechanical model + face assets

A deliverables-only folder (no code). Two STEP files at the root: **ElectronBot.step** (~31.7 MB, the complete electromechanical digital twin — shells, gears, bearings, M2 screws, micros servos, *and* the PCB geometry) and **Box.step** (the FDM-printable box from the end of the video, NOT the electrical base). B-rep/NURBS solids in mm, exported from **Fusion 360** (robot) and **Rhino3D** (box); the editable `.f3d` source is only on an external A360 link.

Emoji/ = an animation state-machine library: 6 emotions (disdain, excited, terrified, anger, sad, static). Each emotion follows the **enter → loop → return** grammar (`_1进入姿势`, `_2可循环动作`, `_3回正`, `_动作演示`)

for splicing expressions; idle adds random blinks and glances.

⚠ **Corrected fact:** the emoji clips are **900×900 H.264, NOT 240×240**. The downscale happens **on the PC** (OpenCV `cv::resize + RGB888`), never on the MCU. Rename files to ASCII before use — the Studio rejects Chinese paths.

5.Docs — Reference store (no markdown)

- **Datasheets/** — 11 PDFs = a chip-level BOM (GC9A01, PAJ7620U2, USB3300, hubs GL850G/SL2.1A/FE8.1/FM116B, STMs). ⚠ The actual F405 (main) and F042 (servo) **have no datasheet here** — fetch them from ST.
- **Images/** — `robot9.png` = the **authoritative servo I2C protocol table**; `robot7.jpg` = the software stack diagram; `robot4.jpg` = the USB topology.
- **_LargeFiles/** — **critical build action:** following `_path.txt`, unzip `msyh.zip` → `Assets/Materials/Fonts/msyh.asset` and `opencv_world348d.zip` → `Assets/Plugins/opencv_world348d.dll` inside the Unity project (excluded by `.gitignore` because of GitHub's 100 MB limit).

The actual narrative documentation lives in the root `README.md` (**Chinese, authoritative**) / `enREADME.md`, not in 5.Docs.

6.Tests — TestDisplayUSB (turnkey bring-up)

The canonical integration test that proves the robot end-to-end (PC → USB → LCD, with the servo path dormant). It ships firmware source + host SDK source + **ready-made binaries** in `_Released/`. The `ElectronBot-fw` here is a snapshot of the canonical firmware from `2.Firmware`. VID/PID match on both sides (0x1001/0x8023). Same frame math: $4 \times (84 \times 512 \text{ B} + 224 \text{ B}) = 172,800 \text{ B}$.

Where to start (build, flash, run)

Fastest path — `6.Tests/TestDisplayUSB`: 1. **Install the USB driver** first: `3.Software/_Tools/BotDriver` (a prerequisite for everything). (*You may need to bind the driver to the app's VID/PID via Zadig.*) 2. **Flash the firmware:** `_Released/STM32F4 Firmware/ElectronBot-fw.hex` via **ST-Link/SWD** (OpenOCD, `stm32f4x`). 3. **Run the demo:** `_Released/Windows EXE/Sample.exe`. Success = the console prints **"Robot connected!"** and `happy.mp4` animates on the face.

To develop: - **Firmware:** `2.Firmware/ElectronBot-fw` (CMake/arm-none-eabi-gcc or the `.ioc` in CubeMX) for the head; `ServoDrive-fw` (CMake) or `ServoDrive-fw-11` (PlatformIO) for the joints. Flash the servo on the **ServoDrive-DK**, then program the production ServoDrive. - **SDK + Unity:** build from `3.Software/SDK` (CMake ≥3.21, MSVC x64). **First**, extract `5.Docs/_LargeFiles` per `_path.txt`. Recreate the

D:\Libraries\Cpp\... paths (OpenCV is in SDK/_Libs/OpenCV_lib.zip). - **Calibrate servos:** flash the ServoToolKit hex, set each servo's ID/PID/limits one at a time, re-flash the normal firmware.

 **The single file to understand the whole system:** 3.Software/SDK/ElectronBotSDK-LowLevel/electron_low_level.cpp — it defines the entire PC↔robot wire protocol; everything else is a wrapper around it.